

[BUG] MottonenStatePreparation decomposition computation fails

<https://github.com/PennyLaneAI/pennylane/issues/4589>

ROW 41

[BUG] MottonenStatePreparation decomposition computation fails #4589

[New issue](#)[Closed](#)

neak11 opened on Sep 12, 2023 · edited by neak11

Edits ...

Expected behavior

The created Pytorch model uses a quantum layer with MottonenStatePreparation that is supposed to be optimized with the provided data.

Actual behavior

The optimization fails. Apparently, the `_get_alpha_y` and `_get_alpha_z` methods from MottonenStatePreparation use `qml.math.take` with `axis=-1`, while the torch implementation of `qml.math.take` seems to only be able to use non-negative axis indices. Negative indices produce the same result as `axis=0`.

Additional information

No response

Source code

```
import pennylane as qml
from pennylane import qnn
import torch
from torch import nn
from torchvision import datasets
from torchvision.transforms import ToTensor
from torch.utils.data import DataLoader

images = datasets.FakeData(size=8000, image_size=(1, 8, 8), num_classes=1, transform=ToTensor())

device = qml.device('default.qubit', wires=6, shots=None)

@qml.qnode(device, interface='torch', diff_method='adjoint')
def circuit(inputs, param):
    norms = torch.norm(inputs, dim=1)
    inputs = inputs/torch.stack([norms for i in range(inputs.size(dim=1))], dim=1)
    qml.MottonenStatePreparation(inputs, wires=range(6))
    qml.BasicEntanglerLayers(param, wires=range(6))
    return qml.expval(qml.PauliY(wires=0))

param_shape = qml.BasicEntanglerLayers.shape(1, 6)
dictionary = {"param": param_shape}
quantum_layer = qnn.TorchLayer(circuit, dictionary)

flat_layer = nn.Flatten(1, -1)
model_net = nn.Sequential(flat_layer, quantum_layer).to("cpu")

dataloader_images = DataLoader(images, batch_size=10, shuffle=True)
losses = nn.L1Loss()
optim = torch.optim.SGD(model_net.parameters(), lr=1e-6)

def training(dataloader, model, lossf, optimizer):
    model.train()
    for batch, (data, labels) in enumerate(dataloader):
        result = model(data)
        loss = lossf(result, labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

Assignees

timmysilv

Labels

[bug](#)

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

[Customize](#) [Subscribe](#)

You're not receiving notifications from this thread.

Participants

```
for ep in range(10):
    training(dataloader_images, model_net, losses, optim)
```

Tracebacks

```
Traceback (most recent call last):
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\test1.py", line 48, in <module>
    training(dataloader_images, model_net, losses, optim)
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\test1.py", line 40, in training
    result = model(data)
              ^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\torch\nn\modules\module.py", line 1518, in __call__
    return forward_call(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\torch\nn\modules\container.py", line 113, in forward
    input = module(input)
            ^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\torch\nn\modules\module.py", line 1518, in __call__
    return forward_call(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\qnn\torch.py", line 45, in forward
    results = self._evaluate_qnode(inputs)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\qnn\torch.py", line 45, in _evaluate_qnode
    res = self.qnode(**kwargs)
          ^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\qnode.py", line 989, in __call__
    res = qml.execute(
          ^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\interfaces\execution.py", line 113, in execute
    tapes = [expand_fn(t) for t in tapes]
             ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\interfaces\execution.py", line 113, in 
    tapes = [expand_fn(t) for t in tapes]
              ^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\interfaces\execution.py", line 113, in expand_fn
    return device.expand_fn(tape, max_expansion=max_expansion)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\_device.py", line 713, in expand_fn
    return self.default_expand_fn(circuit, max_expansion=max_expansion)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\_device.py", line 684, in default_expand_fn
    circuit = _local_tape_expand(
              ^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\_device.py", line 84, in _local_tape_expand
    obj = QuantumScript(obj.decomposition(), _update=False)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\operation.py", line 113, in decomposition
    return self.compute_decomposition(
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\templates\state_preparation.py", line 113, in compute_decomposition
    alpha_y_k = _get_alpha_y(a, len(wires_reverse), k)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\templates\state_preparation.py", line 113, in _get_alpha_y
    numerator = qml.math.take(a, indices=indices_numerator, axis=-1)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\autoray\autoray.py", line 80, in get_lib_fn
    return get_lib_fn(backend, fn)(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages\pennylane\math\single_dispatch.py", line 113, in get_lib_fn
    return tensor[fancy_indices]
           ~~~~~~^~~~~~
IndexError: index 32 is out of bounds for dimension 0 with size 10

Process finished with exit code 1
```

System information

```
Name: PennyLane
Version: 0.32.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/PennyLaneAI/pennylane
Author:
Author-email:
License: Apache License 2.0
Location: C:\Users\XYZ\PycharmProjects\pennylaneToProject\venv\Lib\site-packages
Requires: appdirs, autograd, autoray, cachetools, networkx, numpy, pennylane-lightning, requests, rustworkx, s
```

```
- default.gaussian (PennyLane-0.32.0)
- default.mixed (PennyLane-0.32.0)
- default.qubit (PennyLane-0.32.0)
- default.qubit.autograd (PennyLane-0.32.0)
- default.qubit.jax (PennyLane-0.32.0)
- default.qubit.tf (PennyLane-0.32.0)
- default.qubit.torch (PennyLane-0.32.0)
- default.qutrit (PennyLane-0.32.0)
- null.qubit (PennyLane-0.32.0)
- lightning.qubit (PennyLane-Lightning-0.32.0)
```

Existing GitHub issues

☒ I have searched existing GitHub issues to make sure the issue does not already exist.



neak11 added **bug** on Sep 12, 2023

neak11 changed the title **[BUG]-[BUG] MottonenStatePreparation decomposition computation fails** on Sep 12, 2023



CatalinaAlbornoz on Sep 13, 2023

Contributor ...

Hi [@neak11](#), thank you for opening this issue!



CatalinaAlbornoz assigned [timmysilv](#) on Sep 13, 2023



timmysilv on Sep 16, 2023 - edited by timmysilv

Edits ... Contributor ...

Hi [@neak11](#), thanks for reporting this! I actually noticed something very similar a while back in [#4460](#), but this bug report is highlighting a key component to the issue! I'll open a bug-fix to internally convert an axis value of -1 to the actual ndim count, and that should fix some things.

Unfortunately, there will still be more work needed to close [#4460](#) as broadcasting support for `MottonenStatePreparation` is not trivial. For example, with my torch fix patched in, I'll get the decomposition for $|10\rangle$ and $|11\rangle$ without broadcasting, respectively:

```
>>> data = [[0, 0, 1, 0], [0, 0, 0, 1]]
>>> for d in data:
...     print(qml.MottonenStatePreparation.compute_decomposition(d, wires=[0, 1]))
[RY(array(3.14159265), wires=[0]), CNOT(wires=[0, 1]), CNOT(wires=[0, 1])]
[RY(array(3.14159265), wires=[0]), RY(array(1.57079633), wires=[1]), CNOT(wires=[0, 1]), RY(array(-1.57079633), wires=[1]), CNOT(wires=[0, 1])]
```

As you can see, they are not the same set of operations. This makes broadcasting more complicated, so we get the wrong result:

```
>>> qml.MottonenStatePreparation.compute_decomposition(data, wires=[0, 1])
[RY(array([3.14159265, 3.14159265]), wires=[0]),
 CNOT(wires=[0, 1]),
 CNOT(wires=[0, 1])]
```

Not all PennyLane operators have broadcasting support yet, and unfortunately `qml.MottonenStatePreparation` is one of those. That said, when you provide a `qml.StatePrep` operator as the first operator to a circuit on `default.qubit`, because it's a simulator, it won't compute a decomposition and it will just set the state to the desired input! You can replace `qml.MottonenStatePreparation` with `qml.StatePrep` in your circuit, while we work on enhancing `MottonenStatePreparation` with broadcasting support. Let me know if that works for you!



As you can see, they are not the same set of operations. This makes broadcasting more complicated, so we get the wrong result:

```
>>> qml.MottonenStatePreparation.compute_decomposition(data, wires=[0, 1])
[RY(array([3.14159265, 3.14159265]), wires=[0]),
 CNOT(wires=[0, 1]),
 CNOT(wires=[0, 1])]
```

Not all PennyLane operators have broadcasting support yet, and unfortunately `qml.MottonenStatePreparation` is one of those. That said, when you provide a `qml.StatePrep` operator as the first operator to a circuit on `default.qubit`, because it's a simulator, it won't compute a decomposition and it will just set the state to the desired input! You can replace `qml.MottonenStatePreparation` with `qml.StatePrep` in your circuit, while we work on enhancing `MottonenStatePreparation` with broadcasting support. Let me know if that works for you!



timmysilv mentioned this on Sep 17, 2023

[fix torch take with axis=-1 #4605](#)



neak11 on Sep 17, 2023

Author ...

Using `qml.StatePrep` instead does the job. Thanks for the tip, [@timmysilv](#) !



timmysilv on Sep 17, 2023

Contributor ...

fantastic, glad to hear it 🥳 I'm going to close this issue, and we'll track the addition of broadcasting support for `MottonenStatePreparation` in the issue linked above



timmysilv closed this as **completed** on Sep 17, 2023

timmysilv added a commit that references this issue on Sep 18, 2023

`fix torch take with axis=-1 (#4605)` ...

Verified de55925

mudit2812 added a commit that references this issue on Sep 19, 2023

`fix torch take with axis=-1 (#4605)` ...

b3b490d

Beeeam mentioned this on Oct 30, 2023

[\[BUG\]MottonenStatePreparation.compute_decomposition #4753](#)

timmysilv mentioned this on Jan 2, 2024

[Add broadcasting support for MottonenStatePreparation #4460](#)

fix torch take with axis=-1 #4605

<> Code

Merged timmysilv merged 2 commits into master from torch-take-axis-fix on Sep 18, 2023

Conversation 2 Commits 2 Checks 0 Files changed 3 +12 -0

 timmysilv commented on Sep 17, 2023

Contributor

...

Context:

Basically, if you call `qml.math.take(x, indices=your_indices, axis=2, like="torch")`, it returns `x[:, :, your_indices]` (with `axis` placeholder colons, in this case 2). If `axis` is -1, we would return `x[indices]` (because `[vals] * -1` gives an empty list). That's wrong.

Description of the Change:

`qml.math.take` for torch with `axis=-1` correctly returns `x[..., indices]` instead of `x[indices]`.

Benefits:

`qml.math.take` works with torch and `axis=-1`.

Related GitHub Issues:

First reported in [#4589](#)

👍 1

fix torch take with axis=-1 53cbf57

 github-actions bot commented on Sep 17, 2023

Contributor

...

Hello. You may have forgotten to update the changelog!

Please edit [doc/releases/changelog-dev.md](#) with:

- A one-to-two sentence description of the change. You may include a small working example for new features.
- A link back to this PR.
- Your name (or GitHub username) in the contributors section.

👍 0

changelog 7bcfb69

Reviewers

 lillian542

✓

 mudit2812

✓

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

3 participants

  